《数字图像处理》实验报告八

姓名：顾凯琪	学号：226001624	班级：计算机 2 班

1. 实验名称：图像分割

2. 实验内容与目标

1. 加深对图像分割的理解
2. 掌握常用的边缘检测算法
3. 加深对图像压缩以及质量评价的理解
4. 了解图像压缩的步骤，掌握图像信息熵、质量评价等基本方法（PSNR，RMSE）

3. 实验任务

1．输入并运行例程，观察边缘检测的效果；

2．在例程基础上，实现图像分割，并试比较 Sobel 边缘检测与 LOG、Canny 边缘检测的差异；

3. 以 Evaluation_Reference 图像为参考，对 Evaluation_ToBeCompare 图像进行质量评价（PSNR，RMSE）

4．撰写实验报告。

4. 实验方法与过程简要介绍（请填入下面空白处）

1.边缘检测与图像分割实验

实验目标

通过不同方法（**Sobel、Prewitt、Roberts、LoG、Canny** 等）实现图像的边缘检测，观察不同算法及参数对边缘提取效果的影响;同时了解阈值分割与区域生长分割方法，掌握图像分割的基本原理与应用。

实验方法

1. **Sobel** 边缘检测
   - 读取图像 **image1.tif**。
   - 使用 **edge** 函数分别基于 **Sobel** 算子，采用自动阈值和指定阈值，进行垂直、水平及综合方向的边缘检测。
   - 显示不同阈值和方向下的检测结果。
2. 方向边缘检测
   - 利用自定义滤波掩模计算 **45** 度和**-45** 度方向的边缘响应,设置阈值进行二值化，显示单方向及合成边缘结果。
3. **Prewitt** 边缘检测
   - 读取图像，采用 **Prewitt** 算子，分别进行垂直方向和水平垂直综合边缘检测，使用自动和指定阈值。
4. **Roberts** 边缘检测
   - 类似 **Prewitt**，使用 **Roberts** 算子检测边缘，观察不同阈值下的效

果。

5. **LoG** 和 **Canny** 边缘检测对比
    - 使用默认参数的 **LoG** 和 **Canny** 算子进行边缘检测。
    - 使用手动设置的参数优化边缘检测效果，比较结果差异。
6. 阈值分割
    - 读取图像 **image2.tif**，通过迭代法计算图像的最佳阈值，进行二值化分割，显示结果。
7. 区域生长分割
    - 对 **image1.tif** 进行区域生长分割，设置种子点及阈值。
    - 分别显示种子点图像、阈值测试图像及最终分割结果。

实验现象与分析

- **Sobel** 算子：自动阈值时边缘清晰，但部分细节可能丢失；指定阈值能控制边缘数量，垂直和水平掩模对不同方向边缘响应敏感。
- 方向边缘检测：自定义掩模成功提取 **45** 度及**-45** 度方向边缘，合成图像能更完整地表现复杂边缘结构。
- **Prewitt** 和 **Roberts** 算子：与 **Sobel** 相比，**Prewitt** 边缘检测结果较为平滑，**Roberts** 响应较弱且边缘更细，适合细节检测。
- **LoG** 与 **Canny** 比较：**LoG** 对噪声较敏感，边缘检测结果较粗；**Canny** 具有较好的噪声抑制和边缘连续性，适合细节丰富的图像。
- 阈值分割：迭代法自动确定阈值，二值化效果良好，能将目标区域与背景有效区分。
- 区域生长分割：基于种子点的连通性分析，实现较为精准的区域提取，分割结果符合预期。

2.图像分割与边缘检测实验

实验目标

在现有边缘检测例程基础上，实现图像分割，并比较 **Sobel**、**LoG** 和 **Canny** 三种边缘检测方法的效果差异。

实验方法

[1]. 读取待处理图像（如 **image1.tif**）。

[2]. 使用 **MATLAB** 提供的 **edge** 函数，采用 **Sobel**、**LoG**、**Canny** 三种方法分别进行边缘检测。

[3]. 对边缘检测结果进行可视化比较。

[4]. 利用边缘结果实现简单图像分割：将边缘检测结果作为掩模（**Mask**），从原

图中提取目标区域。
[5]. 观察三种算法在边缘检测精度、连续性和噪声鲁棒性上的差异。

实验现象与分析

- **Sobel** 算子：
  - 计算简单，对噪声较敏感；
  - 边缘检测效果一般，边缘不够细腻；
  - 有时边缘不连续。
- **LoG**（拉普拉斯高斯）算子：
  - 引入高斯滤波，具备一定的平滑能力；
  - 抑制部分噪声，边缘较光滑；
  - 检测结果相比 **Sobel** 更清晰，细节更丰富。
- **Canny** 算子：
  - 多阶段处理，包含平滑、梯度计算、非极大值抑制和双阈值处理；
  - 对噪声鲁棒性强，边缘细且连续；
  - 检测结果最为理想，但计算复杂度较高。

3.图像质量评价

实验目标

基于给定的参考图像 **Evaluation_Reference**，对待评价图像 **Evaluation_ToBeCompare** 进行质量评价，计算其峰值信噪比（**PSNR**）和均方根误差（**RMSE**），以量化图像的失真程度和质量差异。

实验方法

[1]. 读取参考图像 **Evaluation_Reference** 和待比较图像 **Evaluation_ToBeCompare**，确保两者大小一致。
[2]. 将图像转换为双精度格式，方便数值计算。
[3]. 计算均方根误差（**RMSE**）：图像像素误差的平方根均值，用于衡量像素级的误差大小。
[4]. 计算峰值信噪比（**PSNR**），反映图像质量，**PSNR** 越高，图像质量越好。
[5]. 显示计算结果并进行简要分析。

实验现象与分析

- **RMSE** 越小说明两幅图像越相近，误差越小；
- **PSNR** 越大表示待评价图像与参考图像相似度越高，图像失真越小；
- 结合 **RMSE** 和 **PSNR** 指标可以较全面评估图像的质量变化。

5. 实验程序与程序注释（请填入下面空白处）

实验一

```
clc;
clear;
f = imread('image1.tif');

figure;

subplot(2,3,1);
imshow(f);
title('原始图像');

[gv1, tvertical] = edge(f,'sobel','vertical');
subplot(2,3,2);
imshow(gv1);
title('自动阈值垂直 Sobel');

[gv1, thorizontal] = edge(f,'sobel','horizontal');
subplot(2,3,3);
imshow(gv1);
title('自动阈值水平 Sobel');

gv2 = edge(f,'sobel',0.15,'vertical');
subplot(2,3,4);
imshow(gv2);
title('指定阈值垂直 Sobel');

gboth = edge(f,'sobel',0.15);
subplot(2,3,5);
imshow(gboth);
title('指定阈值水平+垂直 Sobel');

[gboth, tboth] = edge(f,'sobel');
subplot(2,3,6);
imshow(gboth);
title('自动阈值水平+垂直 Sobel');
```

实验二

```
w45 = [-2 -1 0; -1 0 1; 0 1 2];
g45 = imfilter(double(f),w45,'replicate');
T45 = 0.3 * max(abs(g45(:)));
```

```
g45_bin = g45 >= T45;

w_45 = [0 1 2; -1 0 1; -2 -1 0];
g_45 = imfilter(double(f), w_45, 'replicate');
T_45 = 0.3 * max(abs(g_45(:)));
g_45_bin = g_45 >= T_45;

figure;

subplot(1,3,1);
imshow(g45_bin);
title('45 度边缘');

subplot(1,3,2);
imshow(g_45_bin);
title('-45 度边缘');

subplot(1,3,3);
imshow(g45_bin + g_45_bin);
title('45 度 + -45 度边缘');
```

实验三

```
clc;
clear;
f = imread('image1.tif');

figure;

subplot(2,2,1);
imshow(f);
title('原始图像');

[gv1, t_auto] = edge(f, 'prewitt', 'vertical');
subplot(2,2,2);
imshow(gv1);
title('自动阈值垂直 Prewitt 边缘');

gv2 = edge(f, 'prewitt', 0.15, 'vertical');
subplot(2,2,3);
imshow(gv2);
title('指定阈值 0.15 垂直 Prewitt 边缘');
```

```
gboth = edge(f, 'prewitt', 0.15);
subplot(2,2,4);
imshow(gboth);
title('指定阈值 0.15 水平+垂直 Prewitt 边缘');
```

实验四

```
clc;
clear;
f = imread('image1.tif');

figure;

subplot(2,2,1);
imshow(f);
title('原始图像');

[gv1, t_auto] = edge(f, 'roberts', 'vertical');
subplot(2,2,2);
imshow(gv1);
title('自动阈值垂直 Roberts 边缘');

gv2 = edge(f, 'roberts', 0.15, 'vertical');
subplot(2,2,3);
imshow(gv2);
title('指定阈值 0.15 垂直 Roberts 边缘');

gboth = edge(f, 'roberts', 0.15);
subplot(2,2,4);
imshow(gboth);
title('指定阈值 0.15 水平+垂直 Roberts 边缘');
```

实验五

```
clc;
clear;
f = imread('image1.tif');

figure;

subplot(3,2,1);
imshow(f);
title('原始图像');
```

```
[g_log_default, tlog] = edge(f, 'log');
subplot(3,2,2);
imshow(g_log_default);
title('[LoG]默认参数边缘检测结果');

[g_canny_default, tc] = edge(f, 'canny');
subplot(3,2,3);
imshow(g_canny_default);
title('[Canny]默认参数边缘检测结果');

[g_log_best, tlog] = edge(f, 'log', 0.003, 2.25);
subplot(3,2,4);
imshow(g_log_best);
title('[LoG]参数 0.003,2.25 边缘检测结果');

[g_canny_best, tc] = edge(f, 'canny', [0.04 0.10], 1.5);
subplot(3,2,6);
imshow(g_canny_best);
title('[Canny]参数[0.04 0.10],1.5 边缘检测结果');
```

实验六

```
clc;
clear;
f = imread('image2.tif');

figure;

subplot(1,2,1);
imshow(f);
title('原始图像');

subplot(1,2,2);
imhist(f);
title('图像直方图');

T = 0.5 * (double(min(f(:))) + double(max(f(:))));
done = false;
while ~done
    g = f >= T;
    Tnext = 0.5 * (mean(f(g)) + mean(f(~g)));
    done = abs(T - Tnext) < 0.5;
    T = Tnext;
```

```
end
disp(['迭代法计算得到的阈值 T = ', num2str(T)]);

figure;
imshow(f <= T);
title('使用迭代方法得到的阈值分割结果');
```

实验七

```
clc;
clear;
f = imread('image1.tif');
if size(f,3) == 3
    f = rgb2gray(f); % 如果是彩色图，转为灰度图
end

figure;
imshow(f);
title('原始图像');

% 设置种子点（用行列坐标，比如第 50 行第 50 列）
seed_row = 50;
seed_col = 50;
seed = sub2ind(size(f), seed_row, seed_col);

threshold = 10; % 阈值

% 调用自定义的区域生长函数
[g, NR, SI, TI] = regiongrow(f, seed, threshold);

% 显示结果
figure;
subplot(2,2,1);
imshow(SI);
title('种子点图像');

subplot(2,2,2);
imshow(TI);
title('所有通过阈值测试的像素');

subplot(2,2,3);
imshow(g);
title('区域生长结果图像');
```

```
% 自定义函数定义
function [g, NR, SI, TI] = regiongrow(I, seed, thresh)
    I = double(I);
    sz = size(I);
    g = false(sz);       % 区域掩码
    NR = 0;              % 区域内像素数
    SI = false(sz);      % 种子点图像
    TI = false(sz);      % 通过阈值测试的像素

    SI(seed) = true;

    stack = seed;        % 像素索引栈，初始为种子点
    g(seed) = true;
    NR = 1;
    seed_val = I(seed);

    while ~isempty(stack)
        idx = stack(end);
        stack(end) = []; % 弹出一个像素索引
        [r, c] = ind2sub(sz, idx);
        for dr = -1:1
            for dc = -1:1
                rr = r + dr;
                cc = c + dc;
                if rr >= 1 && rr <= sz(1) && cc >= 1 && cc <= sz(2)
                    nidx = sub2ind(sz, rr, cc);
                    if ~g(nidx) && abs(I(nidx) - seed_val) <= thresh
                        TI(nidx) = true; % 通过阈值测试
                        g(nidx) = true;
                        NR = NR + 1;
                        stack(end+1) = nidx; % 加入栈
                    end
                end
            end
        end
    end
end
```

实验八

```
clc;
clear;
```

```
f = imread('image1.tif');  % 读取图像
subplot(2,3,1), imshow(f), title('原始图像');

% Sobel 边缘检测
edge_sobel = edge(f, 'sobel');
subplot(2,3,2), imshow(edge_sobel), title('Sobel 边缘图');
sobel_seg = uint8(edge_sobel) .* f;
subplot(2,3,5), imshow(sobel_seg), title('Sobel 分割图');

% LoG 边缘检测
edge_log = edge(f, 'log');
subplot(2,3,3), imshow(edge_log), title('LoG 边缘图');
log_seg = uint8(edge_log) .* f;
subplot(2,3,6), imshow(log_seg), title('LoG 分割图');

% Canny 边缘检测
figure;
subplot(1,3,1), imshow(f), title('原始图像');
edge_canny = edge(f, 'canny');
subplot(1,3,2), imshow(edge_canny), title('Canny 边缘图');
canny_seg = uint8(edge_canny) .* f;
subplot(1,3,3), imshow(canny_seg), title('Canny 分割图');
```

实验九

```
clc;
clear;

% 读取图像（假设图像均为灰度图或单通道图像）
ref = imread('Evaluation_Reference.tif');
cmp = imread('Evaluation_ToBeCompare.tif');

% 转为双精度
ref = double(ref);
cmp = double(cmp);

% 确保图像尺寸相同
if ~isequal(size(ref), size(cmp))
    error('参考图像与待比较图像尺寸不匹配');
end

% 计算 RMSE
rmse = sqrt(mean((ref(:) - cmp(:)).^2));
```

```
% 计算 PSNR
max_pixel = 255;  % 假设 8 位图像
psnr_val = 10 * log10( (max_pixel^2) / (rmse^2) );

% 输出结果
fprintf('RMSE = %.4f\n', rmse);
fprintf('PSNR = %.4f dB\n', psnr_val);

% 可视化（可选）
figure;
subplot(1,2,1), imshow(uint8(ref)), title('参考图像');
subplot(1,2,2), imshow(uint8(cmp)), title('待评价图像');
```

6. 实验结果与总结（请填入下面空白处）

实验一：**MATLAB** 数字图像基本操作

例 **1 Sobel** 边缘检测

实验结果

使用自动阈值的 **Sobel** 算子对图像进行垂直和水平边缘检测，得到清晰的边缘轮廓。指定阈值 **0.15** 时，边缘更加突出且噪声减少，同时综合垂直和水平方向的边缘更完整。



实验总结

Sobel 算子能够较好地检测图像的水平和垂直边缘，自动阈值适用于一般情况，而指定阈值可以更精细地控制边缘检测效果，适合根据需求调整。

例 **2** 方向边缘检测（**45** 度和**-45** 度）

实验结果

通过自定义 **45** 度和**-45** 度滤波掩模分别提取对应方向的边缘，二值化处理后边缘清晰，两个方向的边缘叠加后图像边缘结构更加完整丰富。



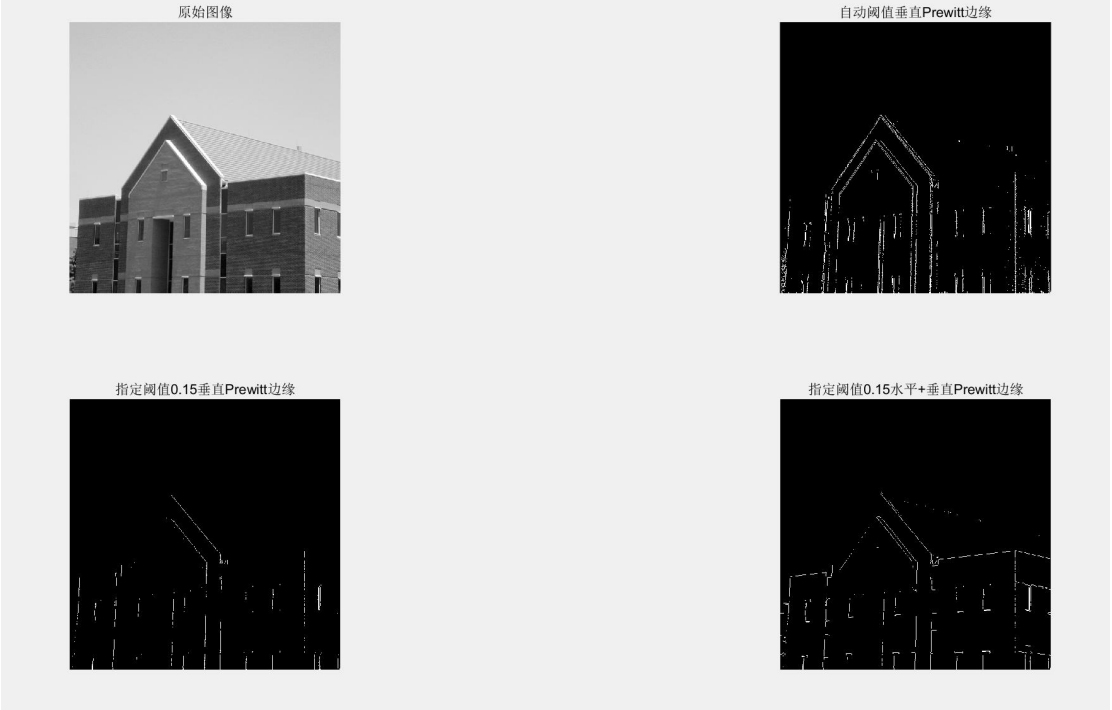
实验总结

使用自定义方向滤波器可针对特定角度提取边缘，有助于捕捉斜向细节，适合对方向敏感的边缘检测任务。

例 **3 Prewitt** 边缘检测

实验结果

Prewitt 算子检测结果较 **Sobel** 算子更平滑，自动阈值下边缘清晰，指定阈值 **0.15** 使边缘更明显且噪声减少，综合方向检测结果较为完整。



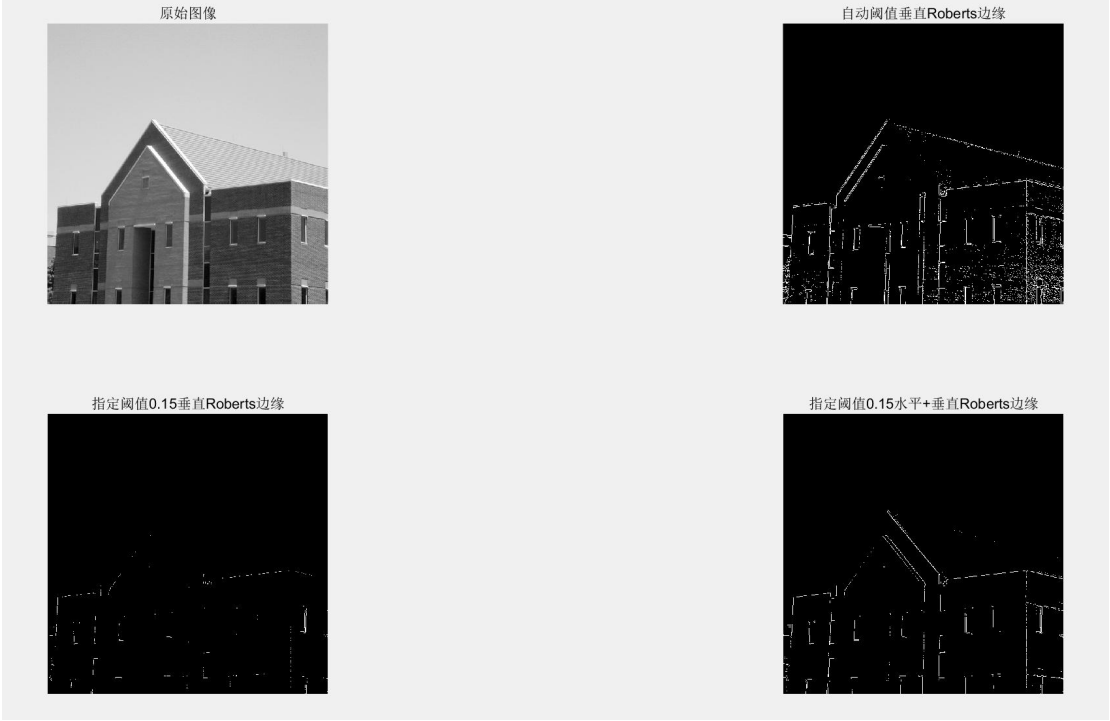
实验总结

Prewitt 算子适合检测图像的基本边缘结构，且响应相对均匀，阈值调整能有效优化边缘提取效果。

例 **4 Roberts** 边缘检测

实验结果

Roberts 算子检测的边缘较细且响应较弱，自动阈值检测边缘数量较多，指定阈值 **0.15** 时边缘更突出，综合方向边缘检测效果明显。



实验总结

Roberts 算子对细节敏感，适合细微边缘提取，但对噪声较敏感，阈值调节对效果影响较大。

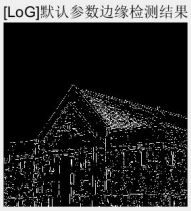
例 **5 LoG** 与 **Canny** 边缘检测对比

实验结果

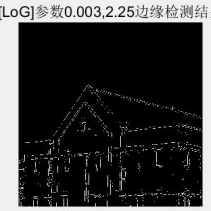
默认参数下，**LoG** 算子检测边缘较粗糙，**Canny** 算子边缘连续且噪声抑制较好。

调参后，两者检测效果有所提升，**Canny** 对细节表现更优。











实验总结

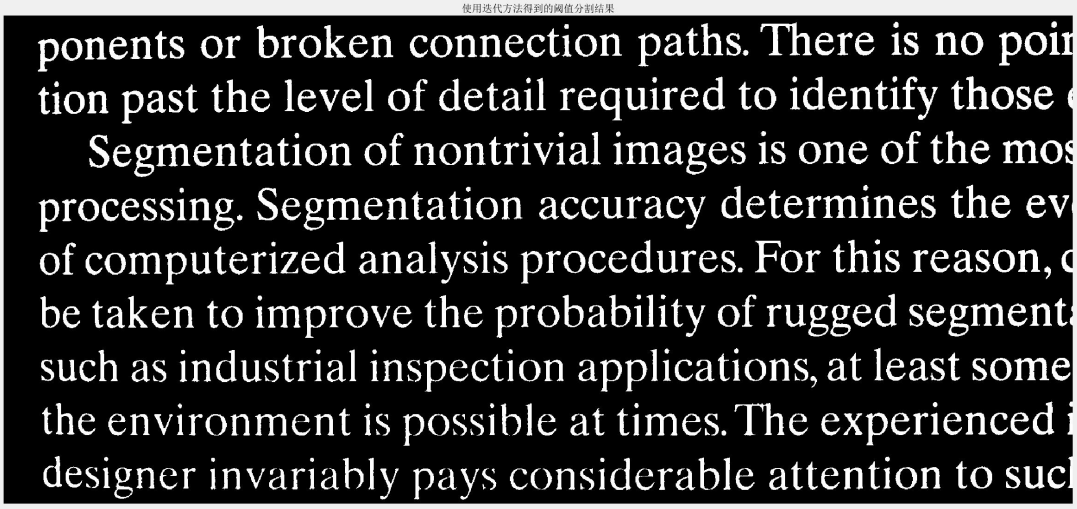
LoG 算子适合检测强边缘但对噪声敏感，**Canny** 算子综合性能优异，适合多场景应用，合理调参能进一步提升效果。

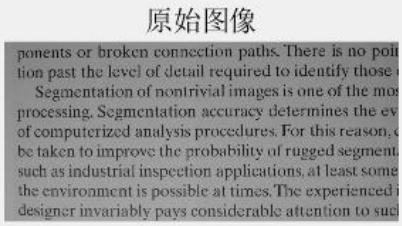
例 **6** 阈值分割

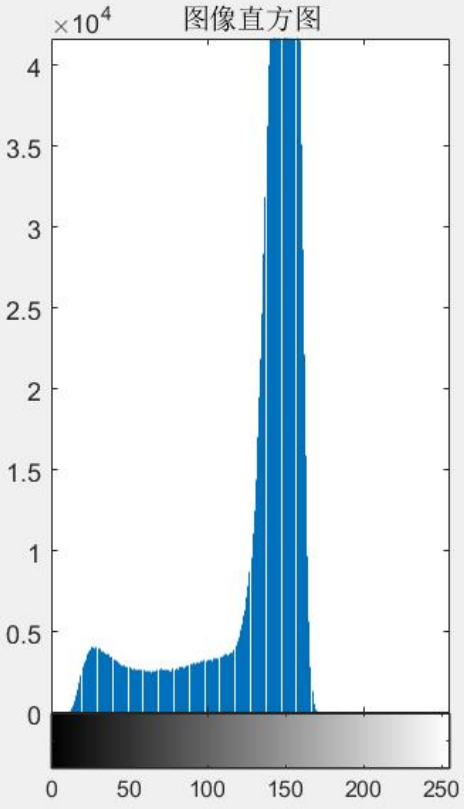
实验结果

通过迭代法自动计算阈值，实现图像二值化分割，目标区域与背景明显分离，分割效果稳定可靠。

使用迭代方法得到的阈值分割结果







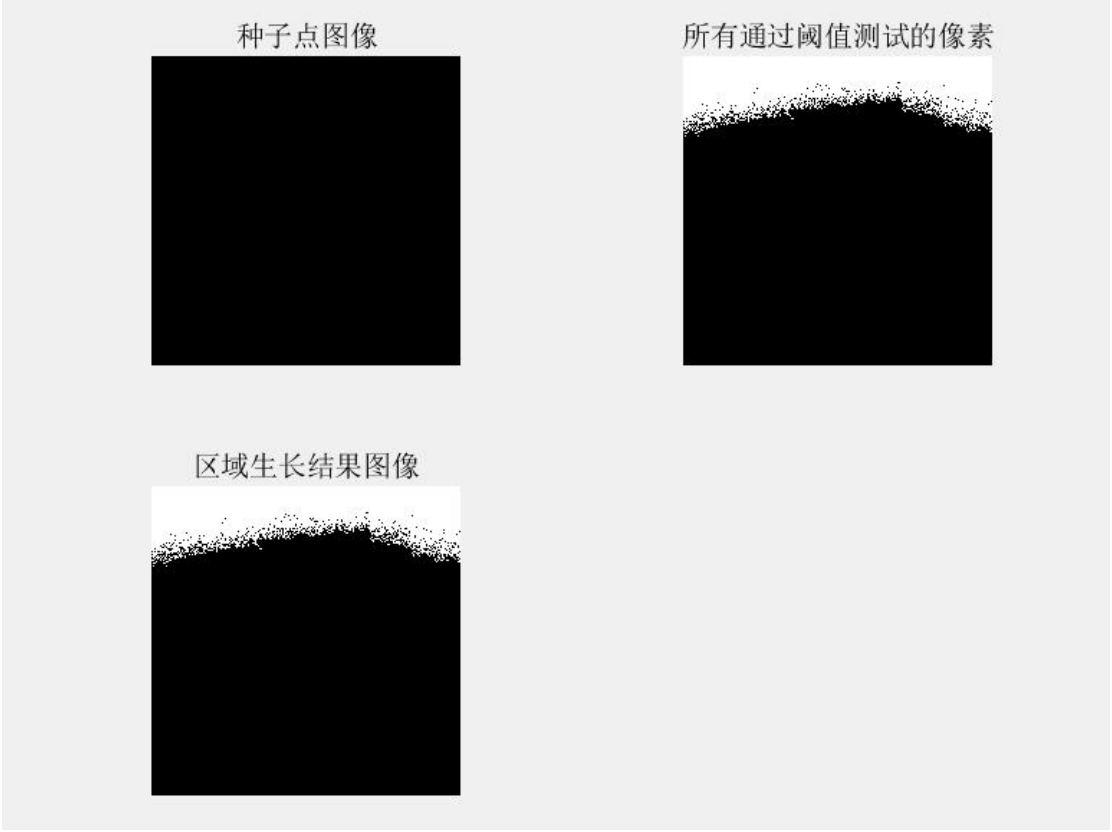
实验总结

迭代阈值法简单有效，适合灰度图像的自动分割，能自适应图像灰度分布，适合预处理和目标提取。

例 **7** 区域生长分割

实验结果

基于设定的种子点和阈值，实现区域连通性生长，准确提取目标区域。种子点图像、阈值测试图像及最终分割结果显示分割过程清晰。



实验总结

区域生长法结合阈值与连通性，有效实现目标区域提取，适合结构较为连续的图像分割，具有较强的可控性和准确性。

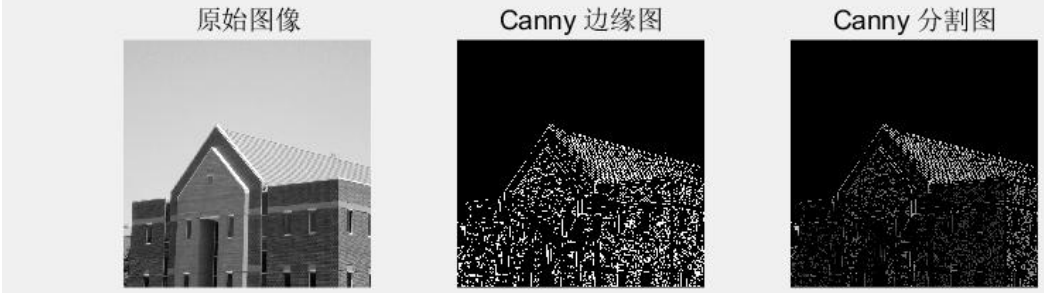
实验二

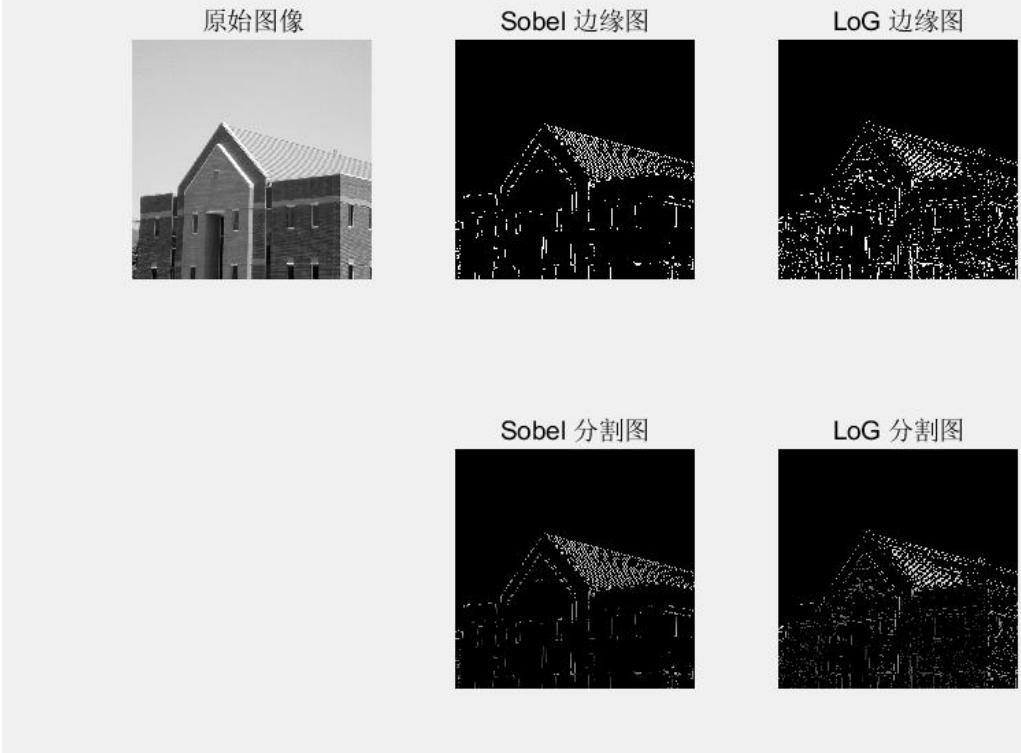
实验结果

三种方法分别实现图像的边缘检测与分割：

- **Sobel**：边缘较粗，断裂多；
- **LoG**：轮廓更连贯，背景抑制较好；

- **Canny**：边缘细腻连续，目标分割清晰，最佳。





实验总结

本实验实现了图像的边缘检测与基于边缘的分割，比较了 **Sobel**、**LoG** 和 **Canny** 三种经典算法的效果：

Sobel 算子实现简单，适用于对边缘精度要求不高的场景；

LoG 算子在平滑与检测之间取得较好平衡；

Canny 算子提供了最佳的边缘检测效果，是多数实际应用中的首选。

该实验加深了对边缘检测在图像分割中的核心作用的理解，也展示了不同算子的特性与适用场景。

实验三

实验结果

由此可见，待比较图像与参考图像存在一定的像素级误差，但 **PSNR** 处于较合理范围，质量总体尚可。

参考图像



待评价图像



实验总结

本实验通过计算 **RMSE** 和 **PSNR** 两个指标对待评价图像的质量进行了定量分析。**RMSE** 提供了误差的绝对量度，**PSNR** 则从信噪比角度衡量图像失真程度。

结合这两者可以有效判断图像处理或传输过程中的质量变化,便于后续优化和改进。